ROUTEEV

**Ruteo inteligente para vehículos eléctricos en Uruguay.**

# RouteEV

RouteEV calcula la ruta más eficiente para un auto eléctrico sobre el grafo vial real de todo el país: revisa la batería tramo a tramo y suma automáticamente las paradas de carga reales que hagan falta en el camino, sobre datos abiertos de OpenStreetMap y de la red de carga pública de UTE Movilidad.

Ruteo inteligente para vehículos eléctricos en Uruguay

> **Estado:** entregable académico (BETA). Se prioriza código claro y bien documentado sobre complejidad de producción — ver `TECHNICAL.md` para el diseño completo y `DEPLOY.md` para la guía de despliegue.

Proyecto de tesis · Uruguay

Generado el 31/07/2026 a partir de `README.md`




## Tabla de contenidos


- El problema
- La solución
- Funcionalidades
- Capturas de pantalla
- Stack tecnológico
- Seguridad
- Cómo correrlo localmente
- Tests y calidad

- Estructura del proyecto
- Documentación completa
- Limitaciones conocidas
- Contexto académico


---

## El problema

La autonomía real de un vehículo eléctrico no es la del folleto: el tráfico, la velocidad y el consumo del propio auto cambian cuánta batería queda al llegar. Los mapas convencionales calculan el camino más corto o más rápido, pero ninguno avisa si esa ruta es la que la batería del vehículo puede hacer, ni dónde parar a cargar si no alcanza.

## La solución

RouteEV construye el camino sobre el **grafo vial real del país** (no una aproximación): cada calle pesa lo que realmente cuesta recorrerla — tiempo, energía, tráfico y peaje — y el sistema revisa la batería antes de cada tramo. Si no alcanza, busca automáticamente una estación de carga real en el camino, sin que el usuario tenga que planearlo.

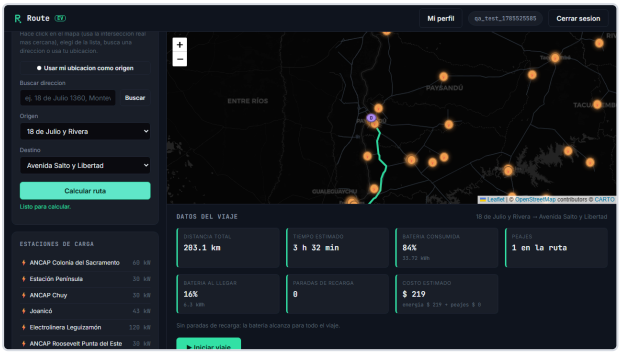
## Funcionalidades

- **Landing pública + login separado del mapa.** `/` presenta el proyecto (el problema, la solución, datos reales del grafo); el mapa y el resto de la app quedan detrás de sesión en `/app`.
- **Grafo real de los 19 departamentos de Uruguay**: ~63.800 intersecciones y ~97.300 tramos de calle desde OpenStreetMap, con Montevideo urbano al detalle de calle residencial y el resto del país a nivel de red troncal, más un radio fino alrededor de cada estación de carga fuera de la capital.
- **209 estaciones de carga reales** de la red pública UTE Movilidad, con tipo de conector, potencia y cantidad de puertos tal cual el listado oficial.
- **Catálogo de 880 modelos de auto eléctrico reales** (14 curados especialmente para Uruguay — Nissan Leaf, Tesla Model 3/Y, Chevrolet Bolt, BYD, VW ID.4, Hyundai Kona, Kia Niro, MG4, Peugeot e-208, entre otros— más 866 de un dataset global de especificaciones EV) que precargan capacidad, consumo y potencia de carga al elegir un modelo. Un campo de búsqueda con autocompletado nativo del navegador (`<input list>` + `<datalist>`, sin dependencias) permite tipear el nombre y encontrarlo al instante entre los 880 — más una búsqueda por IA para identificar cualquier otro modelo por nombre.
- **Dijkstra y A*** sobre un mismo motor de vehículo eléctrico, con verificación de batería arista por arista y recarga automática al pasar por una estación.
- **Batería en porcentaje**, no en kWh: el usuario indica cuánta carga tiene el auto (0–100%) y la app deriva la autonomía real a partir del vehículo elegido.

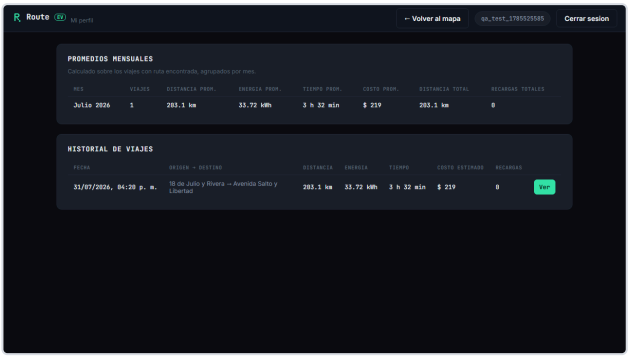
- **Selección de origen/destino** por click en el mapa, lista de intersecciones, dirección escrita a mano (geocodificada vía Nominatim/OpenStreetMap) o la ubicación real del dispositivo.
- **Panel de resultado del viaje**: distancia, tiempo, batería consumida y restante, costo estimado, peajes en la ruta y el detalle completo de cada parada de carga.
- **Seguimiento en vivo del viaje** vía GPS real del navegador ( `watchPosition` ), con un ícono de auto que se mueve pegado a la posición real, proyectando la ubicación sobre la ruta calculada para saber cuánto avanzó, cuánta batería usó y si ya pasó por una recarga.
- **Perfil de usuario**: historial de viajes **realmente recorridos** (no simples cálculos de ruta) y promedios mensuales. Un viaje cuenta al llegar a destino o al cancelar habiendo avanzado una distancia real; si se corta a mitad de camino, queda guardado el tramo efectivamente recorrido, no la ruta completa que se había planeado.
- **Login con usuario/contraseña o Google**, JWT + PBKDF2, historial en SQLite.

## Capturas de pantalla

CÁLCULO DE RUTA CON PARADAS DE CARGA




PERFIL E HISTORIAL DE VIAJES



## Stack tecnológico

| CAPA | TECNOLOGÍA |
|---|---|
| Backend | Python 3.11+, FastAPI, Pydantic, Uvicorn |
| Persistencia | SQLite (sin ORM, SQL explícito) |
| Autenticación | JWT (PyJWT) + PBKDF2-HMAC-SHA256, OAuth "Iniciar sesión con Google" |
| Frontend | HTML/CSS/JavaScript plano (sin build ni framework), Leaflet.js |
| Datos geográficos | OpenStreetMap (Overpass API), Nominatim (geocodificación) |
| Datos de estaciones | UTE Movilidad (listado público oficial) |
| IA opcional | API de Anthropic (Claude) para identificar modelos de auto no catalogados |
| CI / calidad | GitHub Actions, Semgrep (security-audit, OWASP Top 10, secrets), pytest |

Ver `TECHNICAL.md` para el detalle de arquitectura, la referencia completa de la API y los algoritmos.

## Seguridad

RouteEV pasó por una revisión de seguridad dedicada (tests automatizados, Semgrep con 356 reglas de OWASP Top 10 / secrets / auditoría, y revisión manual de autenticación, autorización y consultas SQL). Resumen de las defensas activas:

- **JWT firmado con secreto fuera del repositorio**: en producción el servidor no arranca si falta `JWT_SECRET`.
- **Contraseñas** con PBKDF2-HMAC-SHA256 (260.000 iteraciones), mínimo de 10 caracteres, y login de tiempo constante para no filtrar por temporización qué usuarios existen.
- **Rate limiting** por IP/usuario en login, IA, cálculo de ruta, descarga del grafo y geocodificación.
- **Toda consulta SQL parametrizada** y todo dato de vehículos/historial filtrado por `user_id` en la base (sin referencias directas a objetos de otro usuario).
- **CSP con lista blanca de orígenes**, HSTS en producción, `X-Content-Type-Options`, `X-Frame-Options`, y `/docs` / `/openapi.json` apagados en producción.
- **La ubicación del usuario nunca viaja en una URL** (siempre en el cuerpo de un POST).
- **`textContent` en el frontend** para todo dato ingresado por el usuario (nombre de usuario, nombre de vehículo), evitando XSS por inyección de HTML.

Detalle completo (incluidos los riesgos que quedan abiertos antes de exponer esto a internet) en `TECHNICAL.md` y `DEPLOY.md`.

## Cómo correrlo localmente

Requiere Python 3.11+ (probado con 3.14). No hace falta Node ni build de frontend.

```
# 1. Crear y activar un entorno virtual (Windows / PowerShell)
python -m venv .venv
.venv\Scripts\Activate.ps1

# (Linux/Mac)
python3 -m venv .venv
source .venv/bin/activate

# 2. Instalar dependencias
pip install -r requirements.txt

# 3. Levantar el servidor (desde la raíz del proyecto)
python -m uvicorn backend.app.main:app --reload
```

Abrir `http://127.0.0.1:8000/` en el navegador. El grafo semilla y las estaciones de carga ya están incluidos en `backend/data/`; la base SQLite se crea sola en el primer arranque. **El primer arranque tarda alrededor de un minuto**: el grafo (~63.800 nodos) se arma entero en memoria al importar `state.py`.

No hace falta configurar nada para correr local — el secreto de firma de tokens se genera solo y queda gitignoreado. Para las funciones opcionales (login con Google, búsqueda de vehículos por IA) o para desplegar, copiar `.env.example` a `.env` y completarlo. **Antes de desplegar, leer `DEPLOY.md`.**

Flujo típico: crear cuenta en `/login` → crear un vehículo (elegir modelo real o cargar datos a mano) → elegir origen/destino (click, lista, dirección o ubicación) → "Calcular ruta" → "Iniciar viaje" para seguir el trayecto en el mapa → "Mi perfil" para ver el historial.

## Tests y calidad

```
pytest
```

18 tests (~4 minutos), cubriendo dos frentes:

- **`test_algorithms.py`** (6 tests): Dijkstra y A* devuelven el mismo costo óptimo sobre el grafo real, con y sin restricción de batería (forzando recarga); la heurística nunca sobreestima el costo real; las rutas devueltas son caminos válidos del grafo.
- **`test_security.py`** (12 tests): regresión de las defensas de seguridad — el tipo de cosa que se rompe en silencio en un refactor porque quitarla no hace fallar ninguna funcionalidad visible.

`.github/workflows/security.yml` corre Semgrep y la suite completa de tests en cada push y cada lunes a las 6:00 UTC (para agarrar reglas nuevas aunque el código no cambie).

## Estructura del proyecto

```
routesd/
├── backend/
│   ├── app/
│   │   ├── graph.py          # Node/Edge/Graph, costo compuesto de arista, carga de datos
│   │   ├── ev_logic.py       # Haversine, consumo energetico, tiempo de recarga
│   │   ├── dijkstra.py       # Dijkstra + logica EV (demostracion en el docstring)
│   │   ├── astar.py          # A* + logica EV + demostracion de admisibilidad de h(n)
│   │   ├── route_result.py   # Formato de resultado compartido por ambos algoritmos
│   │   ├── auth.py           # PBKDF2 + JWT + verificacion de Google ID token
│   │   ├── database.py       # SQLite (usuarios, vehiculos, historial)
│   │   ├── geocoding.py      # Direccion escrita -> coordenadas, via Nominatim
│   │   ├── ai_lookup.py      # "Cualquier auto por nombre" via Claude (opcional)
│   │   ├── rate_limit.py     # Topes de uso por IP/usuario
│   │   ├── schemas.py        # Modelos Pydantic de entrada/salida de la API
│   │   ├── config.py         # Pesos del costo compuesto, JWT, limites, entorno
│   │   ├── state.py          # Grafo cargado una vez al arrancar
│   │   └── routes/           # Endpoints de FastAPI (auth, graph, vehicles, route)
│   ├── data/                 # Grafo, estaciones y catalogo de EVs (generados)
│   ├── scripts/               # Descarga/generacion de datos (OSM, UTE, geocoding)
│   └── tests/                # test_algorithms.py, test_security.py
├── frontend/
│   ├── landing.html / login.html / app.html / profile.html
│   ├── css/
│   └── js/                    # api.js, landing.js, login.js, map.js, profile.js
├── docs/screenshots/          # Capturas usadas en este README
├── .github/workflows/security.yml
├── TECHNICAL.md                # Documento tecnico: arquitectura, API, algoritmos
├── DEPLOY.md                   # Guia de puesta en produccion
├── requirements.txt
└── pytest.ini
```

## Documentación completa

- `PRODUCT.md` — el proyecto como producto: problema, propuesta de valor, usuarios objetivo, alcance funcional, posicionamiento frente a alternativas, decisiones de producto con su razón, y qué deliberadamente no hace.
- `TECHNICAL.md` — arquitectura, flujo end-to-end, referencia completa de la API REST, algoritmos (Dijkstra/A*, función de costo, demostración de admisibilidad de la heurística), modelo de datos y detalle de seguridad.

- `DEPLOY.md` — números medidos de arranque/memoria/latencia, dónde desplegar y qué falta cerrar antes de exponer esto a internet.

## Limitaciones conocidas

- El monto de los peajes es un placeholder ( `ENERGY_WEIGHT_MIN_PER_KWH` / `TOLL_PENALTY_MIN` en `config.py` ): se detecta el tramo con peaje real de OpenStreetMap, pero la tarifa en pesos todavía no está cargada para la mayoría de los peajes.
- Los datos técnicos de las 209 estaciones son reales; la **coordenada** de las que no se pudieron geocodificar por dirección exacta queda a nivel de ciudad o departamento (ver `geocode_precision` en `charging_stations.json` ).
- Fuera de Montevideo el grafo solo cubre vías principales, no ruteo calle a calle dentro de otras ciudades.
- El tamaño del grafo es el límite de escala actual: ~56 s de arranque y ~104 MB de RAM por proceso (medido y documentado en `DEPLOY.md` ).

Detalle completo, con las razones de diseño detrás de cada una, en `TECHNICAL.md` .

## Contexto académico

Este proyecto es un entregable de tesis/trabajo final de carrera: implementa y compara Dijkstra y A* como algoritmos de ruteo sobre un grafo vial real con restricción de batería, con foco en que el diseño (costo compuesto, espacio de estados, demostración de admisibilidad de la heurística) esté documentado y sea defendible ante un tribunal, no solo funcional. El detalle de cada decisión de diseño vive en `TECHNICAL.md` y en los docstrings de `backend/app/dijkstra.py` / `backend/app/astar.py` .